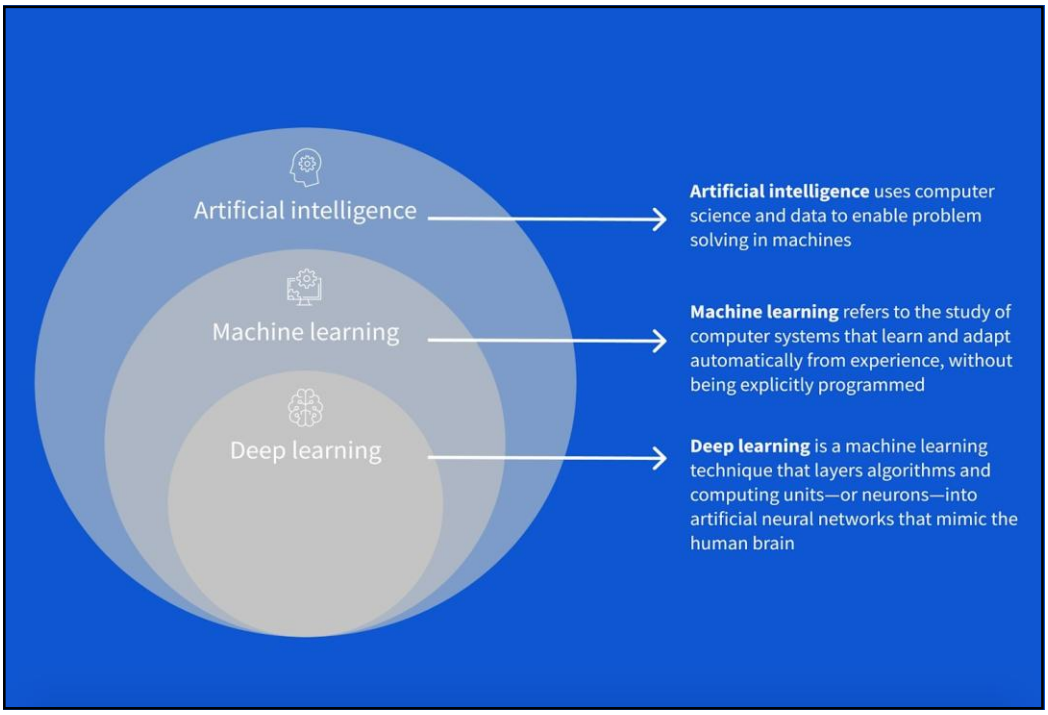


What is AI, ML, DL

- **What is AI?**
  - AI is a branch of computer science by which **we can create intelligent machines which can behave like a human, think like humans, and be able to make decisions.**

- **What is ML?**
  - Machine learning (ML) is a type of AI that allows software applications to **become more accurate at predicting outcomes without being explicitly programmed to do so.**

- **What is DL?**
  - Deep Learning is a type of machine learning based on artificial neural networks in which **multiple layers of processing are used to extract higher level features from data.**

DI and ML

- **Limitation of ML or why do we need DL over ML?**
  - ML is not useful while working with **high dimensionality of data.**
  - ML cannot solve crucial AI problems like **NLP, image recognition etc..**
  - Feature extractions (**ML cannot automatically generate features)**
- **Solution of ML limitations is Deep learning.**
  - Deep learning models are capable of focusing on the right features by themselves, requiring little guidance from programmers.
  - These models also partially solve the dimensionality problem.

Deep Learning (DL)

- Deep learning is implemented through Neural Networks.
- DL is a subset of ML that uses artificial neural networks with multiple layers (hence "deep") to learn representations of data.
- Deep learning is about training models to learn complex patterns in data through artificial neural networks
- It has been particularly successful in areas such as image recognition, natural language processing, autonomous driving and speech recognition.
- In computer science engineering, DL involves studying advanced neural network architectures such as convolution neural networks (CNNs), recurrent neural networks (RNNs), and transformer models. It also requires knowledge of optimization techniques like gradient descent, back propagation, and regularization.
- Unlike traditional machine learning algorithms, which often require manual feature engineering, deep learning models automatically learn representations of data through multiple layers of processing.
-

What is Deep Learning?

Deep learning is a class of machine learning algorithms that:

- Use a cascade of multiple layers of nonlinear processing units for feature extraction and transformation. Each successive layer uses the output from the previous layer as input.
- Learn in supervised (e.g., classification) and/or unsupervised (e.g., pattern analysis) manners.
- Learn multiple levels of representations that correspond to different levels of abstraction; the levels form a hierarchy of concepts.

The major characteristics of Deep Learning include:

1. **Neural Networks:** These artificial neural networks are inspired by the structure and function of the human brain and are made up of interconnected nodes called neurons. These layers allow the network to process and transform input data to produce output.

1. **Deep Neural Networks:** Traditional neural networks consist of only one or two hidden layers, whereas Deep Learning models comprise multiple hidden layers which enable them to learn complex data representations. They perform exceptionally in tasks like speech recognition, Natural Language Processing (NLP) and more.

1. **End-to-end Learning:** Deep learning models can perform end-to-end learning by learning the entire task from raw input data without relying on any intermediate representations. This simplifies the learning process and eliminates the need for developing any explanatory representations.

1. **Training with Backpropagation:** Deep learning models undergo training through the backpropagation algorithm, a process that entails iteratively modifying the model's weights by assessing the disparity between predicted and actual outputs. This iterative optimization procedure serves to minimize errors and enhance the overall performance of the model.

1. **Feature Learning:** Relevant features from raw data can be automatically extracted by Deep learning models. This is significantly advantageous as they reduce the need for manual feature engineering. When dealing with large and high-dimension datasets, this becomes even more beneficial for businesses.

Why Deep Learning?

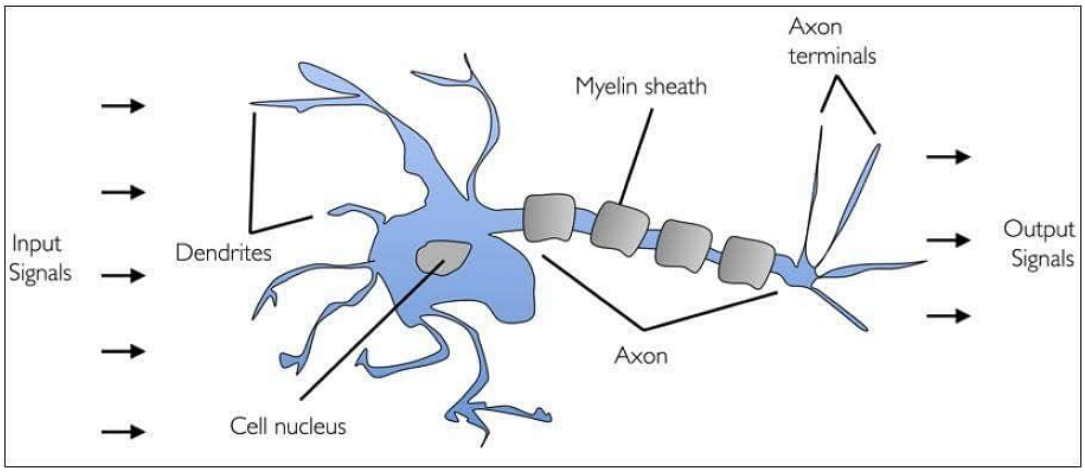
- Limitations of traditional machine learning algorithms
  - Not good at handling high dimensional data.
  - Difficult to do feature extraction and object recognition.
- Advantages of deep learning
  - DL is computationally expensive, but it is capable of handling high dimensional data.
  - Feature extraction is done automatically.

Artificial Intelligence (AI)	Machine Learning (ML)	Deep Learning (DL)
AI simulates human intelligence to perform tasks and make decisions.	ML is a subset of AI that uses algorithms to learn patterns from data.	DL is a subset of ML that employs artificial neural networks for complex tasks.
AI may or may not require large datasets; it can use predefined rules.	ML heavily relies on labeled data for training and making predictions.	DL requires extensive labeled data and performs exceptionally with big datasets.
AI can be rule-based, requiring human programming and intervention.	ML automates learning from data and requires less manual intervention.	DL automates feature extraction, reducing the need for manual engineering.
AI can handle various tasks, from simple to complex, across domains.	ML specializes in data-driven tasks like classification, regression, etc.	DL excels at complex tasks like image recognition, natural language processing, and more.
AI algorithms can be simple or complex, depending on the application.	ML employs various algorithms like decision trees, SVM, and random forests.	DL relies on deep neural networks, which can have numerous hidden layers for complex learning.
AI may require less training time and resources for rule-based systems.	ML training time varies with the algorithm complexity and dataset size.	DL training demands substantial computational resources and time for deep networks.
AI systems may offer interpretable results based on human rules.	ML models can be interpretable or less interpretable based on the algorithm.	DL models are often considered less interpretable due to complex network

		architectures.
AI is used in virtual assistants, recommendation systems, and more.	ML is applied in image recognition, spam filtering, and other data tasks.	DL is utilized in autonomous vehicles, speech recognition, and advanced AI applications.

Biological Neuron

A human brain has billions of neurons. Neurons are interconnected nerve cells in the human brain that are involved in processing and transmitting chemical and electrical signals. Dendrites are branches that receive information from other neurons.



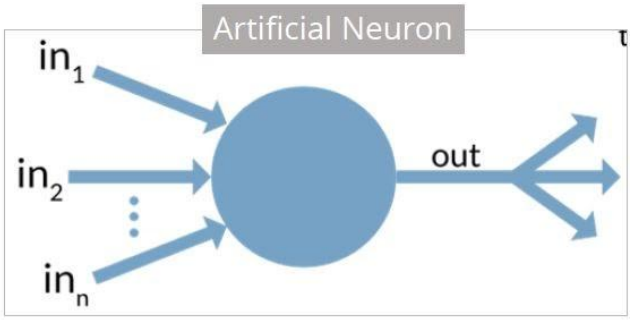
Cell nucleus or Soma processes the information received from dendrites. Axon is a cable that is used by neurons to send information. Synapse is the connection between an axon and other neuron dendrites.

Rise of Artificial Neurons (Based on Biological Neuron)

Researchers Warren McCullock and Walter Pitts published their first concept of simplified brain cell in 1943. This was called McCullock-Pitts (MCP) neuron. They described such a nerve cell as a simple logic gate with binary outputs.
Multiple signals arrive at the dendrites and are then integrated into the cell body, and, if the accumulated signal exceeds a certain threshold, an output signal is generated that will be passed on by the axon. In the next section, let us talk about the artificial neuron.

What is Artificial Neuron

An artificial neuron is a mathematical function based on a model of biological neurons, where each neuron takes inputs, weighs them separately, sums them up and passes this sum through a nonlinear function to produce output.



Biological Neuron vs. Artificial Neuron

The biological neuron is analogous to artificial neurons in the following terms:

Biological Neuron	Artificial Neuron
Cell Nucleus (Soma)	Node
Dendrites	Input
Synapse	Weights or interconnections
Axon	Output

Artificial Neuron at a Glance

The artificial neuron has the following characteristics:

- A neuron is a mathematical function modeled on the working of biological neurons
- It is an elementary unit in an artificial neural network
- One or more inputs are separately weighted
- Inputs are summed and passed through a nonlinear function to produce output
- Every neuron holds an internal state called activation signal
- Each connection link carries information about the input signal
- Every neuron is connected to another neuron via connection link

Neuron: A Generalized Unit in Neural Networks

- A **neuron**, or artificial neuron, is a more generalized version of the perceptron and is the building block of modern deep learning architectures. Neurons in deep learning are part of **multi-layer neural networks**, which can have multiple hidden layers.
- Neurons are part of more sophisticated architectures called **multi-layer perceptrons (MLPs)** or deep neural networks, where neurons are organized into layers (input layer, hidden layers, and output layer). Each layer performs computations, and the output of one layer is fed as input to the next.
- **Continuous Output**: Neurons can output continuous values, unlike the binary output of a perceptron. This makes them more versatile for tasks like regression, multi-class classification, and complex feature extraction.
- **Learning through Back propagation**: Neurons in deep learning models are trained using **back propagation** and gradient descent, which adjusts the weights based on the error between the predicted and actual outputs. The perceptron uses a simpler update rule that works only for linearly separable problems.

Neural Networks

- Neural networks are a type of machine learning model that is based on the structure and function of the human brain.
- They consist of interconnected nodes, or neurons, that process and transmit information.
- The basic building block of a neural network is a neuron, which takes input values and produces an output value.
- Neurons are connected to other neurons in the network through weighted connections, which determine the strength of the connection between neurons.
- Neural networks can be represented as a series of layers, with each layer consisting of a set of neurons that perform a specific function.
- The input layer receives input data, such as images or text, and passes it through the network. The output layer produces the final output of the network, such as a classification or regression result.
- Between the input and output layers, there can be one or more hidden layers.
- These layers perform intermediate computations on the input data, gradually transforming it into a form that can be used by the output layer.
- The number of hidden layers and the number of neurons in each layer can be adjusted to optimize the performance of the network for a given task.
-

In addition to the layers and connections, neural networks also have activation functions, which determine the output of each neuron based on its input. Common activation functions include sigmoid, tanh, and ReLU

Structure of Neural Networks

Neural networks are primarily used in AI to train computers to process data like a human brain does. It is a system of interconnected nodes, or "neurons," organized in layers: Input, hidden and output layer. In the input layer, the network receives the data. The hidden layer performs computations and transformations on the input data. And the output layer produces the final output.

The neural networks can learn efficiently by adjusting the weights and biases of their neurons based on error of certain predictions. This is termed as training and involves two of the key steps- forward propagation and back propagation.

Deep learning isn't just a theoretical concept rather it's driving real-world innovation. Let us look at some of the real-world applications of deep learning-

How Neural Networks Work: The Basics

A neural network is a computational model inspired by the human brain's network of neurons. It consists of layers of interconnected nodes, also called neurons, where each connection between nodes has a weight that adjusts during training.

Key Components of Neural Networks:

1. **Neurons (Nodes)**: Each neuron processes input data, applies a mathematical function, and passes the result to the next layer of neurons.
2. **Layers**: A typical neural network has three types of layers:**Input Layer**: Takes in the raw data.**Hidden Layers**: Intermediate layers where the network learns to transform the input data into meaningful representations. A deep network has multiple hidden layers, allowing it to learn more abstract features.**Output Layer**: Produces the final prediction or classification result.
3. **Weights and Biases**: Each connection between neurons has a weight that determines the strength of the signal passed between them. The model also has biases that adjust the output of neurons to better fit the data.
4. **Activation Function**: After a neuron processes the input, an activation function determines whether the signal should pass through to the next layer. Common activation functions include ReLU (Rectified Linear Unit), sigmoid, and tanh.
5. **Loss Function**: The loss function measures the difference between the model's prediction and the actual result, guiding the training process to minimize this error.
6. **Optimizer**: The optimizer adjusts the weights and biases in the network using techniques like **gradient descent** to minimize the loss function.

Training a Neural Network

Training a neural network involves feeding data through the network, calculating the error (or loss), and using optimization algorithms (such as gradient descent) to adjust the weights to reduce that error. This process is repeated iteratively across many epochs (passes through the dataset) until the model converges to a solution.

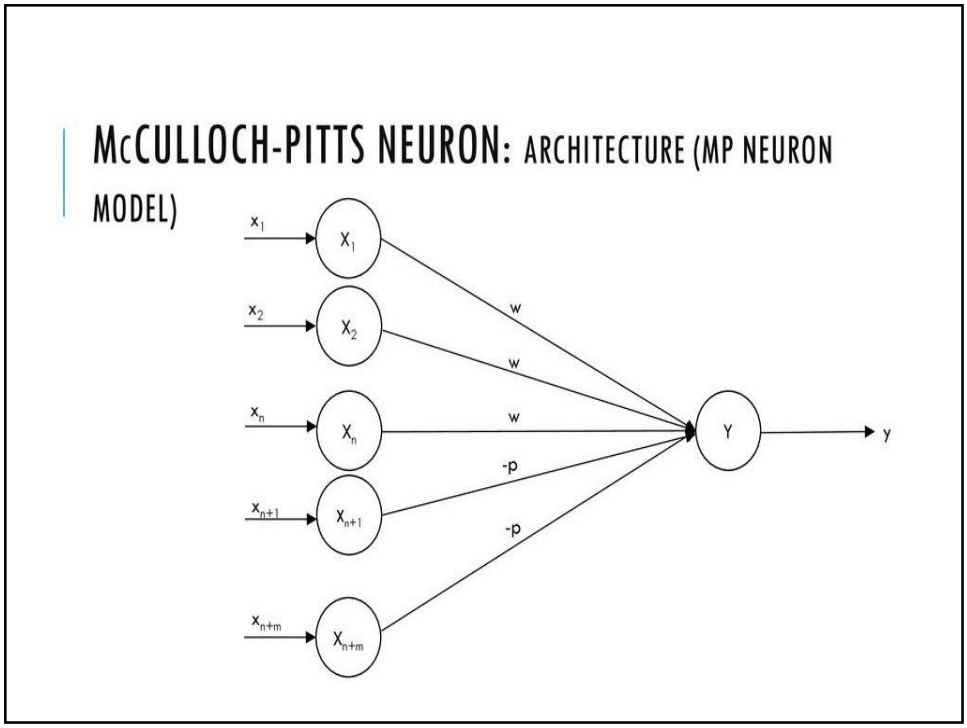
Types of Neural Networks

Neural networks come in various architectures, each suited for different types of tasks. Here are a few common types:

1. **Feedforward Neural Networks (FNN)**:The simplest type of neural network, where information flows in one direction from input to output. FNNs are commonly used for tasks like regression and classification.
2. **Convolutional Neural Networks (CNN)**:CNNs are specifically designed for image-related tasks. They use convolutional layers that apply filters to the input data, allowing the model to learn spatial hierarchies and detect patterns like edges, textures, and objects. CNNs are widely used in image recognition, object detection, and video analysis.
3. **Recurrent Neural Networks (RNN)**:RNNs are designed for sequential data, where the output depends not just on the current input but also on previous inputs (e.g., time series or language). RNNs are used in applications like speech recognition, language modeling, and machine translation.
4. **Long Short-Term Memory Networks (LSTM)**:LSTMs are a special kind of RNN that helps the model remember long-term dependencies in sequences, addressing the problem of vanishing gradients in traditional RNNs. They are often used for tasks that require understanding context over time, such as language translation or sentiment analysis.

5. **Generative Adversarial Networks (GANs)**:GANs consist of two networks: a generator that creates new data and a discriminator that evaluates it. They are often used to generate realistic images, videos, and even music by training the generator to fool the discriminator. GANs have gained popularity in creative applications like art and deepfakes.

McCulloch-Pitts Neuron Model



- It is also called MP Neuron model
- It is the first computational model of a neuron proposed by Warren McCulloch and Walter Pitts in 1943
- The model allows binary 0 or 1 only.
- These binary neurons are connect by directed weights path
- The connected path can have positive weights (Excitatory) or negative weights (Inhibitory)
- There will be same weights for the Excitatory connection entering into particular neuron
- The neuron is associated with a threshold value.
- The neuron activates (fires) if the total inputs to the neuron is greater than or equal to the threshold
- The MP neuron model has no particular training algorithm
- The MP neurons are used as building blocks on which we can model any function or phenomenon, which can be represented as a logic function

- It is divided into 2 parts (g,f). The first part g take the inputs, perform an aggregation and based on aggregation value the second part f takes the decision

- g(x1,x2,x3,…xn-1,xn)=g(x) $= \sum_{i=1}^{m} wixi$

$$y = \begin{cases} 1, & \text{if } \sum_{i=1}^{m} w_i x_i \geq t, \\ 0, & \text{otherwise} \end{cases}$$

y =f(g(x)

- *Let $[x_1, x_2, \ldots, x_m]$ be a vector of input signals where each x_i has a value of 1 or 0.*
- *Let $[w_1, w_2, \ldots, w_m]$ be a vector of weights corresponding to the input signals where each w_i has a value of 1, −1 or 0.*
  - *Input signals with a weight of 1 are called excitatory since they contribute towards a positive output signal in the sum.*
  - *Input signals with a weight of −1 are called inhibitory since they repress a positive output signal in the sum.*
  - *Input signals with a weight of 0 do not contribute at all to the neuron.*
- *Then for some threshold value t, an integer, the output signal is determined by the following activation function:*

$$y = \begin{cases} 1, & \text{if } \sum_{i=1}^{m} w_i x_i \geq t, \\ 0, & \text{otherwise} \end{cases}$$

- *The neuron is said to be "activated" when the weighted sum is greater than the threshold value.*

The MCP Neuron is illustrated below. Note that the x_i input signals are analogous to the *dendrites*, the activation function is analogous to the *soma* and the output is analogous to the *axon*.

Calculation of threshold manually in MP Neuron model

- No training in MP neuron model
- No Weight & bias adjustments in MP Neuron model
- Theta has to be calculated manually
- Activation function y = 1 if netweighted sum >= theta
  = 0 other wise

  Another Activation function

  Y = 1 if netweighted sum > 0
  0 if netweighted sum = 0
  -1 if netweighted sum < 0

For ex.

X1	X2	W1	W2	Desired output	Net-sum	Output estimated
0	0	1	1	0	0x1+0x1 =0	0
0	1	1	1	0	0x1+1x1=1	1

1	0	1	1	1	1x1+0x1=1	1
1	1	1	1	0	1x1+1x1=2	2

Assume w1=1 and w2 =1

In this case we want neuron to trigger only when the input is (1,0). So now we need to calculate the theta value to satisfy activation function. If theta is >= 1 then the AF triggers for input (0,1), (1,0) and (1,1). But we need to trigger only for input (1,0). No way we can get theta value which satisfies AF.

Therefore we change the weights w1= 1 and w2 = -1

X1	X2	W1	W2	Desired output	Net-sum	Output estimated
0	0	1	-1	0	0x1+0x-1 =0	0
0	1	1	-1	0	0x1+1x-1=-1	-1
1	0	1	-1	1	1x1+0x-1=1	1
1	1	1	-1	0	1x1+1x-1=0	0

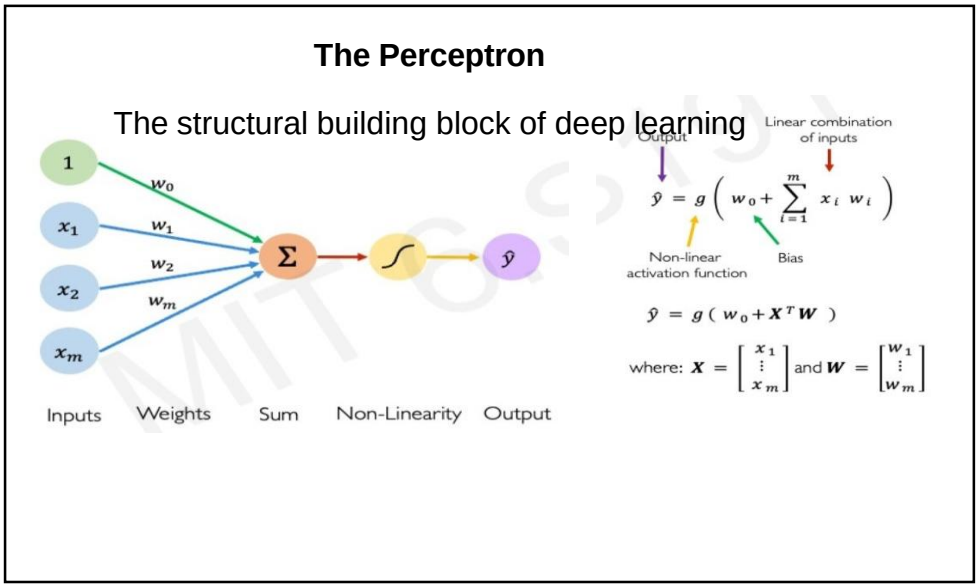
If theta is >= 1 then only for input (1,0) it triggers

What is Perceptron

Perceptron was introduced by Frank Rosenblatt in 1957. He proposed a Perceptron learning rule based on the original MCP neuron. A Perceptron is an algorithm for supervised learning of binary classifiers. This algorithm enables neurons to learn and processes elements in the training set one at a time.

It is a neural network link that contains computations to track features and uses Artificial Intelligence in the input data. This neural links to the artificial neurons using simple logic gates with binary outputs. An artificial neuron invokes the mathematical function and has node, input, weights, and output equivalent to the cell nucleus, dendrites, synapse, and axon, respectively, compared to a biological neuron.

Perceptron - The Simplest Neural Unit



- Perceptron learning is a type of supervised learning algorithm in machine learning that is used for binary classification tasks. It is a feed forward network. i.e data moves only in one direction. It is used for binary classification.
- A binary classifier in machine learning is a type of model that is trained to classify data into one of two possible categories, typically represented as binary labels such as 0 or 1, true or false, or positive or negative.
- The perceptron is a single-layer neural network that takes input values and produces an output based on a set of weights and a bias term.
- The algorithm updates the weights and bias term based on the training examples provided until the model can correctly classify all examples or reaches a maximum number of iterations.
- The perceptron learning algorithm works by taking a set of input values and producing an output based on the dot product of the input values and the corresponding weights.
- The output is then passed through an activation function (usually a step function) to produce a binary output, either 0 or 1.
- During training, the algorithm adjusts the weights and bias terms based on the errors made by the perceptron.
- If the perceptron incorrectly classifies an example, the weights and bias terms are adjusted to reduce the error.
- This process continues until the perceptron can correctly classify all examples or a maximum number of iterations is reached.
- The perceptron learning algorithm has limitations and is only effective for linearly separable data.
- It cannot be used for non-linearly separable data, which requires more advanced algorithms such as support vector machines or neural networks.
- However, the perceptron learning algorithm is useful for simple binary classification tasks and can provide a good starting point for more complex machine learning problems.
- A single perceptron can only be used to implement **linearly separable** functions. It takes both real and boolean inputs and associates a set of **weights** to them, along with a **bias** (the

threshold).

Perceptron is a type of artificial neural network, which is a fundamental concept in machine learning. The basic components of a perceptron are:

1. Input Layer: The input layer consists of one or more input neurons, which receive input signals from the external world or from other layers of the neural network.
2. Weights: Each input neuron is associated with a weight, which represents the strength of the connection between the input neuron and the output neuron.
3. Bias: A bias term is added to the input layer to provide the perceptron with additional flexibility in modeling complex patterns in the input data.
4. Activation Function: The activation function determines the output of the perceptron based on the weighted sum of the inputs and the bias term. Common activation functions used in perceptrons include the step function, sigmoid function, and ReLU function.
5. Output: The output of the perceptron is a single binary value, either 0 or 1, which indicates the class or category to which the input data belongs.
6. Training Algorithm: The perceptron is typically trained using a supervised learning algorithm such as the perceptron learning algorithm or backpropagation. During training, the weights and biases of the perceptron are adjusted to minimize the error between the predicted output and the true output for a given set of training examples.
7. Overall, the perceptron is a simple yet powerful algorithm that can be used to perform binary classification tasks and has paved the way for more complex neural networks used in deep learning today.

Advantages and Disadvantages of Perceptron

Here are some advantages and disadvantages of perceptrons:

Advantages:

- Perceptrons are simple and easy to understand. They can be trained using a straight forward algorithm known as the perceptron learning rule.
- Perceptrons can be used for binary classification problems where the decision boundary is linear. They are particularly useful for problems that involve separating data into two classes.
- Perceptrons are computationally efficient and can make predictions quickly once they are trained.

Disadvantages:

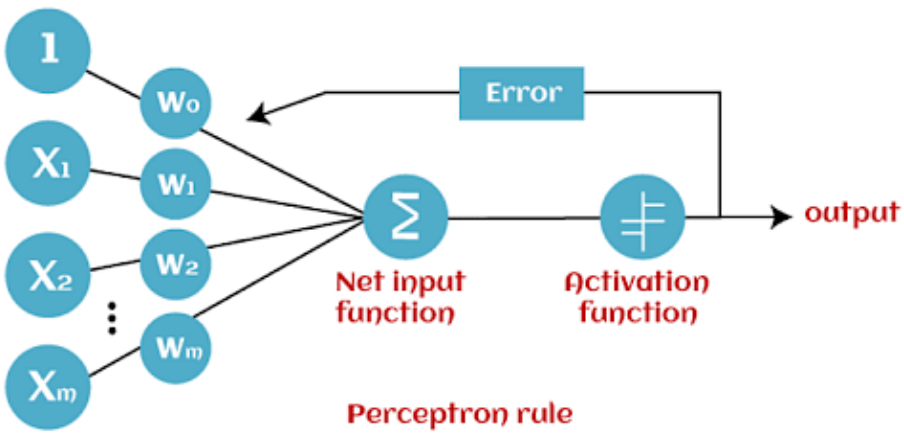
- Perceptrons are limited to linearly separable problems. They cannot solve problems that require non-linear decision boundaries.
- Perceptrons are prone to over-fitting if the data is noisy or if there are too many input variables relative to the size of the training dataset.

Comparison between MP Neuron Model and Perceptron Model

- Both, MP Neuron Model as well as the Perceptron model work on linearly separable data.
- MP Neuron Model only accepts boolean input whereas Perceptron Model can process any real input.
- Inputs aren't weighted in MP Neuron Model, which makes this model less flexible. On the other hand, Perceptron model can take weights with respective to inputs provided.
- While using both the models we can adjust threshold input to make the model fit our the dataset.
- MP-Neuron – Input and Output values = Binary values
- Perceptron Input = Real Value, Output = Binary value

How Does Perceptron Work?

AS discussed earlier, Perceptron is considered a single-layer neural link with four main parameters. The perceptron model begins with multiplying all input values and their weights, and then adds these values to create the weighted sum. Further, this weighted sum is applied to the activation function 'f' to obtain the desired output. This activation function is also known as the step function and is represented by 'f.'



This step function or Activation function is vital in ensuring that output is mapped between (0,1) or (-1,1). Take note that the weight of input indicates a node's strength. Similarly, an input value gives the ability the shift the activation function curve up or down.
Step 1: Multiply all input values with corresponding weight values and then add to calculate the weighted sum. The following is the mathematical expression of it:

$\sum wi*xi = x1*w1 + x2*w2 + x3*w3 + \ldots\ldots x4*w4$

Add a term called bias 'b' to this weighted sum to improve the model's performance.

Step 2: An activation function is applied with the above-mentioned weighted sum giving us an output either in binary form or a continuous value as follows:

$Y = f(\sum wi*xi + b)$

Types of Perceptron models

We have already discussed the types of Perceptron models in the Introduction. Here, we shall give a more profound look at this:

1. Single Layer Perceptron model: One of the easiest ANN(Artificial Neural Networks) types consists of a feed-forward network and includes a threshold transfer inside the model. The main objective of the single-layer perceptron model is to analyze the linearly separable objects with binary outcomes. A Single-layer perceptron can learn only linearly separable patterns.

2. Multi-Layered Perceptron model: It is mainly similar to a single-layer perceptron model but has more hidden layers.

Forward Stage: From the input layer in the on stage, activation functions begin and terminate on the output layer.

Backward Stage: In the backward stage, weight and bias values are modified as per the model's requirement. The backstage removed the error between the actual output and demands originating backward on the output layer. A multilayer perceptron model has a greater processing power and can process linear and non-linear patterns. Further, it also implements logic gates such as AND, OR, XOR, XNOR, and NOR.

Advantages:

- A multi-layered perceptron model can solve complex non-linear problems.
- It works well with both small and large input data.
- Helps us to obtain quick predictions after the training.
- Helps us obtain the same accuracy ratio with big and small data.

Disadvantages:

- In multi-layered perceptron model, computations are time-consuming and complex.
- It is tough to predict how much the dependent variable affects each independent variable.
- The model functioning depends on the quality of training.

Build and Train Single Layer Perceptron Model

A Single Layer Perceptron is the simplest type of neural network. It consists of a single layer of neurons connected to inputs. The perceptron model works by applying a linear function to the input data (a weighted sum) followed by an activation function (e.g., a step or sigmoid function).

1. Initialize Parameters: Set weights and bias.
2. Forward Pass: Compute the weighted sum of inputs.
3. Activation: Use a step function to determine the output.
4. Training: Adjust weights using the Perceptron Learning Rule based on errors.

Binary Classification on a Linearly Separable Dataset

Binary classification involves predicting one of two classes (e.g., 0 or 1). For a dataset to be linearly separable, a straight line (or hyperplane in higher dimensions) can perfectly separate the two classes.

- Example: Classifying red and blue dots on a 2D plane with a straight line.
- A perceptron can handle such data because it is inherently linear. Training the perceptron adjusts the weights to find the optimal line that separates the two classes.

Characteristics of the Perceptron Model

The following are the characteristics of a Perceptron Model:

1. It is a machine learning algorithm that uses supervised learning of binary classifiers.
2. In Perceptron, the Optimal weight coefficient is automatically learned.
3. Initially, weights are multiplied with input features, and then the decision is made whether the neuron is fired or not.
4. The activation function applies a step rule to check whether the function is more significant than zero.
5. The linear decision boundary is drawn, enabling the distinction between the two linearly separable classes +1 and -1.
6. If the added sum of all input values is more than the threshold value, it must have an output signal; otherwise, no output will be shown.

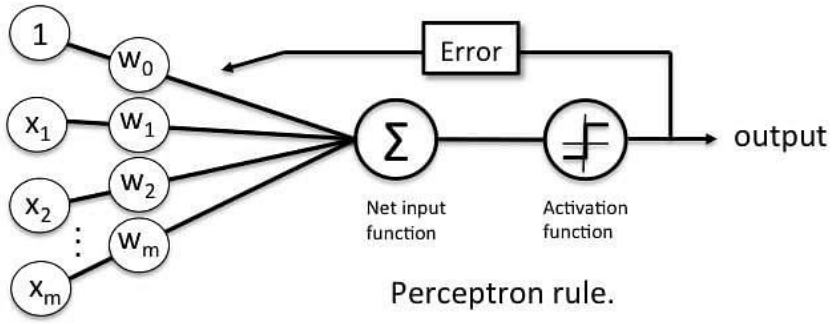
Limitation of Perceptron Model

The following are the limitation of a Perceptron model:

1. The output of a perceptron can only be a binary number (0 or 1) due to the hard-edge transfer function.
2. It can only be used to classify the linearly separable sets of input vectors. If the input vectors are non-linear, it is not easy to classify them correctly.

Perceptron Learning Rule

Perceptron Learning Rule states that the algorithm would automatically learn the optimal weight coefficients. The input features are then multiplied with these weights to determine if a neuron fires or not.



Perceptron rule.

The Perceptron receives multiple input signals, and if the sum of the input signals exceeds a certain threshold, it either outputs a signal or does not return an output. In the context of supervised learning and classification, this can then be used to predict the class of a sample.

Perceptron Function

Perceptron is a function that maps its input "x," which is multiplied with the learned weight coefficient; an output value "f(x)"is generated.

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\sum_{i=1}^{m} w_i x_i$$

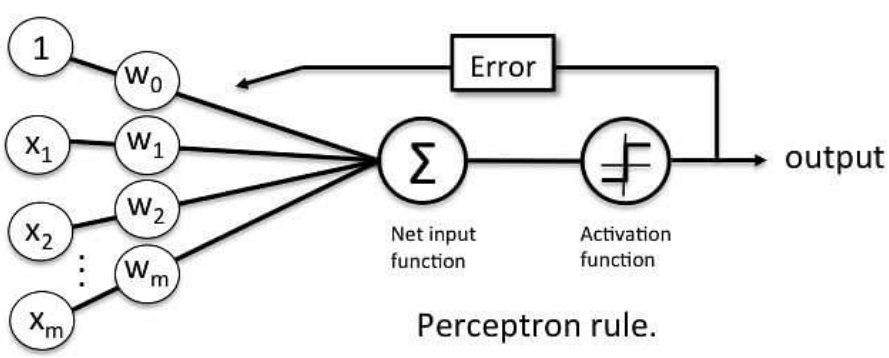
In the equation given above:

- "w" = vector of real-valued weights
- "b" = bias (an element that adjusts the boundary away from origin without any dependence on the input value)
- "x" = vector of input x values
- "m" = number of inputs to the Perceptron

The output can be represented as "1" or "0." It can also be represented as "1" or "-1" depending on which activation function is used.

Inputs of a Perceptron

A Perceptron accepts inputs, moderates them with certain weight values, then applies the transformation function to output the final result. The image below shows a Perceptron with a Boolean output.

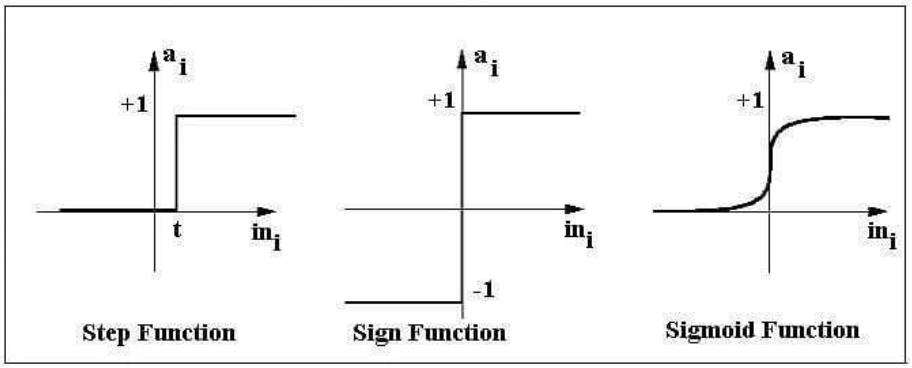


A Boolean output is based on inputs such as salaried, married, age, past credit profile, etc. It has only two values: Yes and No or True and False. The summation function "∑" multiplies all inputs of "x" by weights "w" and then adds them up as follows:

$$w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

Activation Functions of Perceptron

The activation function applies a step rule (convert the numerical output into +1 or -1) to check if the output of the weighting function is greater than zero or not.



For example:

If $\sum w_i x_i > 0$ => then final output "o" = 1 (issue bank loan)
Else, final output "o" = -1 (deny bank loan)

Step function gets triggered above a certain value of the neuron output; else it outputs zero. Sign Function outputs +1 or -1 depending on whether neuron output is greater than zero or not. Sigmoid is the S-curve and outputs a value between 0 and 1.

Output of Perceptron

Perceptron with a Boolean output:
Inputs: x1…xn
Output: o(x1….xn)

$$o(x_1, \ldots, x_n) = \begin{cases} 1 \text{ if } w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n > 0 \\ -1 \text{ otherwise} \end{cases}$$

Weights: wi=> contribution of input xi to the Perceptron output;
w0=> bias or threshold
If $\sum w.x > 0$, output is +1, else -1. The neuron gets triggered only when weighted input reaches a certain threshold value.

$$o(\vec{x}) = sgn(\vec{w} \cdot \vec{x})$$

$$sgn(y) = \begin{cases} 1 \text{ if } y > 0 \\ -1 \text{ otherwise} \end{cases}$$

An output of +1 specifies that the neuron is triggered. An output of -1 specifies that the neuron did not get triggered.
"sgn" stands for sign function with output +1 or -1.

Error in Perceptron

In the Perceptron Learning Rule, the predicted output is compared with the known output. If it does not match, the error is propagated backward to allow weight adjustment to happen.
Let us discuss the decision function of Perceptron in the next section.
Perceptron: Decision Function
A decision function φ(z) of Perceptron is defined to take a linear combination of x and w vectors.

$$\boldsymbol{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

The value z in the decision function is given by:

$$z = w_1 x_1 + \ldots + w_m x_m$$

The decision function is +1 if z is greater than a threshold θ, and it is -1 otherwise.

$$\phi(z) = \begin{cases} 1 & if\ z \geq \theta \\ -1 & otherwise \end{cases}$$

This is the Perceptron algorithm.

Bias Unit

For simplicity, the threshold θ can be brought to the left and represented as w_0x_0, where $w_0 = -\theta$ and $x_0 = 1$.

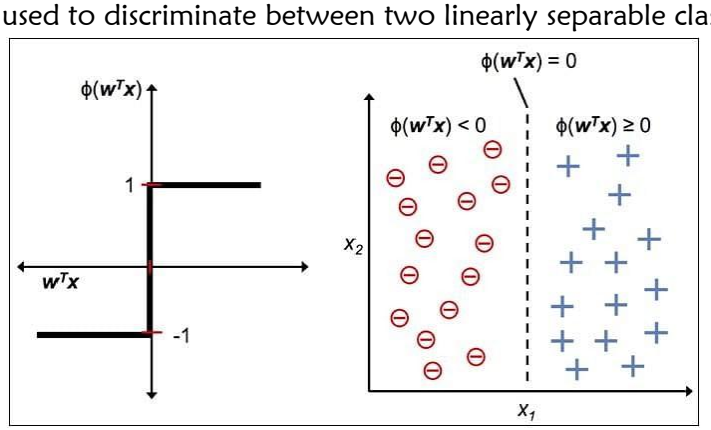
$$z = w_0 x_0 + w_1 x_1 + \ldots + w_m x_m = \boldsymbol{w}^T \boldsymbol{x}$$

The value w0 is called the bias unit.
The decision function then becomes:

$$\phi(z) = \begin{cases} 1 & if\ z \geq 0 \\ -1 & otherwise \end{cases}$$

Output:

The figure shows how the decision function squashes w^Tx to either +1 or -1 and how it can be used to discriminate between two linearly separable classes.

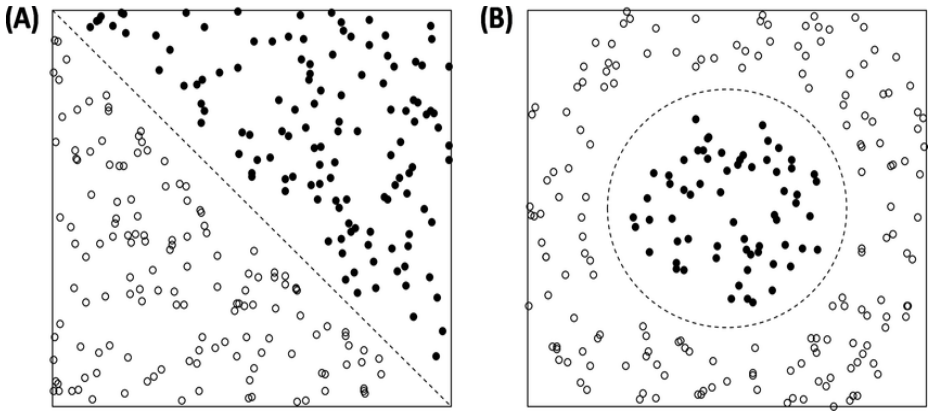


Types of activation functions include the sign, step, and sigmoid functions.

Implement Logic Gates with Perceptron

Perceptron - Classifier Hyperplane

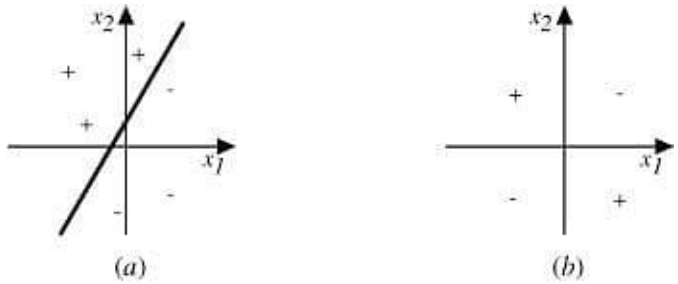
The Perceptron learning rule converges if the two classes can be separated by the linear hyperplane. However, if the classes cannot be separated perfectly by a linear classifier, it could give rise to errors.
Linearly separable: If you can classify the data set into different categories or classes using single straight line in 2 dimensional or a hyperplane in higher dimensions.
Linearly non-separable – the data st may contain multiple classes and requires non-linear line for separating them into respective classes.



As discussed the classifier boundary for a binary output in a Perceptron is represented by the equation given below:

$$\vec{w} \cdot \vec{x} = 0$$

The diagram above shows the decision surface represented by a two-input Perceptron.



Observation:

- In Fig(a) above, examples can be clearly separated into positive and negative values; hence, they are linearly separable. This can include logic gates like AND, OR, NOR, NAND.
- Fig (b) shows examples that are not linearly separable (as in an XOR gate).
- Diagram (a) is a set of training examples and the decision surface of a Perceptron that classifies them correctly.
- Diagram (b) is a set of training examples that are not linearly separable, that is, they cannot be correctly classified by any straight line.
- X1 and X2 are the Perceptron inputs.

What is Logic Gate?

Logic gates are the building blocks of a digital system, especially neural networks. In short, they are the electronic circuits that help in addition, choice, negation, and combination to form complex circuits. Using the logic gates, Neural Networks can learn on their own without you having to manually code the logic. Most logic gates have two inputs and one output.

Each terminal has one of the two binary conditions, low (0) or high (1), represented by different voltage levels. The logic state of a terminal changes based on how the circuit processes data.

Based on this logic, logic gates can be categorized into seven types:

- AND
- NAND
- OR
- NOR
- NOT
- XOR
- XNOR

Implementing Basic Logic Gates With Perceptron

The logic gates that can be implemented with Perceptron are discussed below.

1. AND

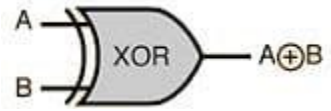
If the two inputs are TRUE (+1), the output of Perceptron is positive, which amounts to TRUE.
This is the desired behavior of an AND gate.
x1= 1 (TRUE), x2= 1 (TRUE)
w0 = -.8, w1 = 0.5, w2 = 0.5
=> o(x1, x2) => -.8 + 0.5*1 + 0.5*1 = 0.2 > 0

2. **OR Function Using A Perceptron**

If either of the two inputs are TRUE (+1), the output of Perceptron is positive, which amounts to TRUE.
This is the desired behavior of an OR gate.
x1 = 1 (TRUE), x2 = 0 (FALSE)
w0 = -.3, w1 = 0.5, w2 = 0.5
=> o(x1, x2) => -.3 + 0.5*1 + 0.5*0 = 0.2 > 0

3. XOR

A XOR gate, also called as Exclusive OR gate, has two inputs and one output.



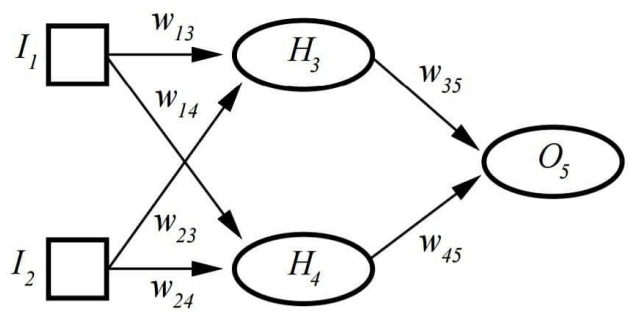
The gate returns a TRUE as the output if and ONLY if one of the input states is true.

XOR Truth Table

Input		Output
A	B	
0	0	0
0	1	1
1	0	1
1	1	0

XOR Gate with Neural Networks

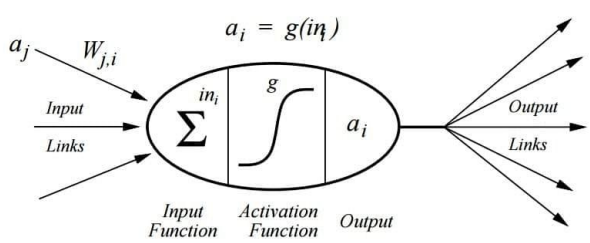
Unlike the AND and OR gate, an XOR gate requires an intermediate hidden layer for preliminary transformation in order to achieve the logic of an XOR gate.



An XOR gate assigns weights so that XOR conditions are met. It cannot be implemented with a single layer Perceptron and requires Multi-layer Perceptron or MLP.
H represents the hidden layer, which allows XOR implementation.
I1, I2, H3, H4, O5are 0 (FALSE) or 1 (TRUE)
t3= threshold for H3; t4= threshold for H4; t5= threshold for O5
H3= sigmoid (I1*w13+ I2*w23–t3); H4= sigmoid (I1*w14+ I2*w24–t4)
O5= sigmoid (H3*w35+ H4*w45–t5);

Sigmoid Activation Function

The diagram below shows a Perceptron with sigmoid activation function. Sigmoid is one of the most popular activation functions.



$$a_i = g(\sum_j W_{j,i} a_j)$$

A Sigmoid Function is a mathematical function with a Sigmoid Curve ("S" Curve). It is a special case of the logistic function and is defined by the function given below:

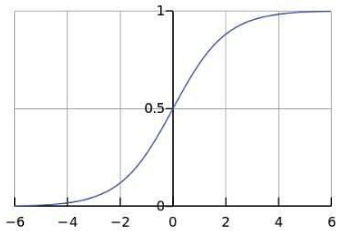
$$\phi_{logistic}(z) = \frac{1}{1+e^{-z}}$$

Here, value of z is:

$$z = w_0 x_0 + w_1 x_1 + \cdots + w_m x_m = \sum_{i=0}^{m} w_i x_i = w^T x$$

Sigmoid Curve

The curve of the Sigmoid function called "S Curve" is shown here.



This is called a logistic sigmoid and leads to a probability of the value between 0 and 1.

This is useful as an activation function when one is interested in probability mapping rather than precise values of input parameter t.

The sigmoid output is close to zero for highly negative input. This can be a problem in neural network training and can lead to slow learning and the model getting trapped in local minima during training. Hence, hyperbolic tangent is more preferable as an activation function in hidden layers of a neural network.

In Summary

- An artificial neuron is a mathematical function conceived as a model of biological neurons, that is, a neural network.
- A Perceptron is a neural network unit that does certain computations to detect features or business intelligence in the input data. It is a function that maps its input "x," which is multiplied by the learned weight coefficient, and generates an output value "f(x).
- "Perceptron Learning Rule states that the algorithm would automatically learn the optimal weight coefficients.
- Single layer Perceptrons can learn only linearly separable patterns.
- Multilayer Perceptron or feedforward neural network with two or more layers have the greater processing power and can process non-linear patterns as well.
- Perceptrons can implement Logic Gates like AND, OR, or XOR.

Perceptron uses Stochastic Gradient Descent to learn the set of weight that minimizes the distance between the misclassified points and the decision boundary. Once Stochastic Gradient Descent converges, the dataset is separated into two regions by a linear hyperplane.

Although it was said the Perceptron could represent any circuit and logic, the biggest problem was that it couldn't represent the XOR gate, *exclusive OR*, where the gate only returns *1* if the inputs are different.

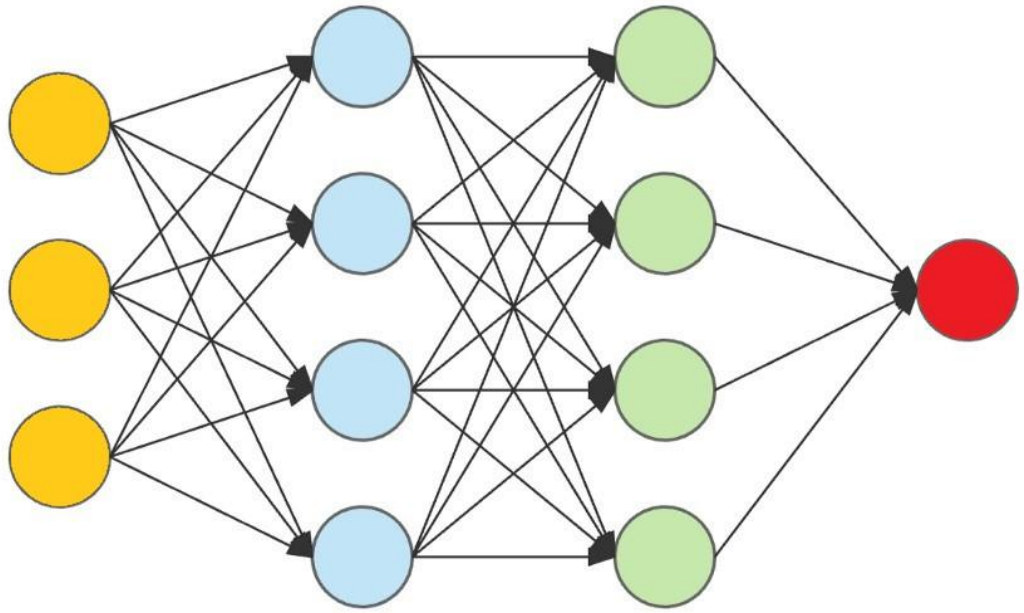
Multilayer Perceptron

The **Multilayer Perceptron** was developed to tackle this limitation. It is a neural network where the mapping between inputs and output is non-linear.

A Multilayer Perceptron has input and output layers, and one or more **hidden layers** with many neurons stacked together. And while in the Perceptron the neuron must have an activation function that imposes a threshold, like ReLU or sigmoid, neurons in a Multilayer Perceptron it can use any arbitrary activation function.

It's important to note that while perceptrons were influential in the development of artificial neural networks, they are limited to learning linearly separable patterns.

However, by stacking multiple perceptrons together in layers and incorporating non-linear activation functions, neural networks can overcome this limitation and learn more complex patterns.



Perceptrons are often used as the building blocks for more complex neural networks, such as multi-layer perceptrons (MLPs) or deep neural networks (DNNs).
By combining multiple perceptrons in layers and connecting them in a network structure, these models can learn and represent complex patterns and relationships in data, enabling tasks such as image recognition, natural language processing, and decision making.
Multilayer Perceptron falls under the category of feedforward algorithms, because inputs are combined with the initial weights in a weighted sum and subjected to the activation function, just like in the Perceptron. But the difference is that each linear combination is propagated to the next layer.
Each layer is *feeding* the next one with the result of their computation, their internal representation of the data. This goes all the way through the hidden layers to the output layer.
But it has more to it.
If the algorithm only computed the weighted sums in each neuron, propagated results to the output layer, and stopped there, it wouldn't be able to *learn* the weights that minimize the cost function. If the algorithm only computed one iteration, there would be no actual learning.

Feed Forward Neural Network

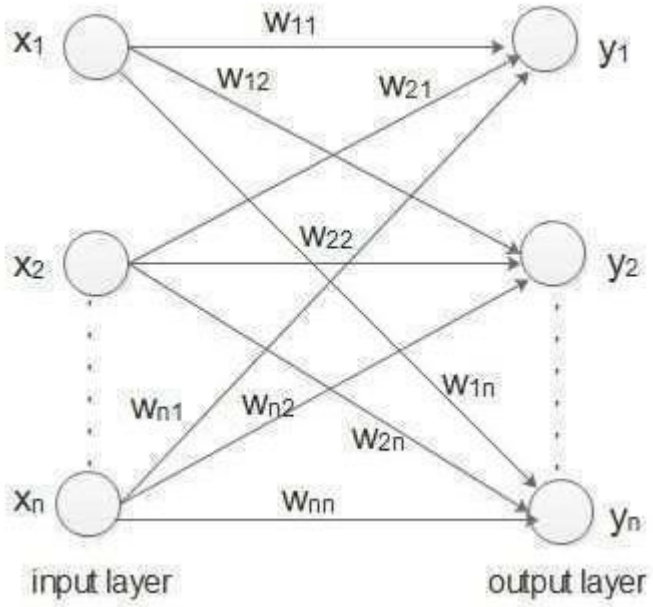
A feed forward neural network is an artificial neural network wherein connections between

the units do not form a cycle. As such, it is different from recurrent neural networks.
The feed forward neural network was the first and simplest type of artificial neural network devised. In this network, the information moves in only one direction, forward, from the input nodes, through the hidden nodes (if any) and to the output nodes.

These are the simplest form of ANNs, where information flows in one direction, from input to output. There are no cycles or loops in the network architecture. Multilayer perceptrons (MLP) are a type of feedforward neural network.

Single-Layer Feed Froward Network

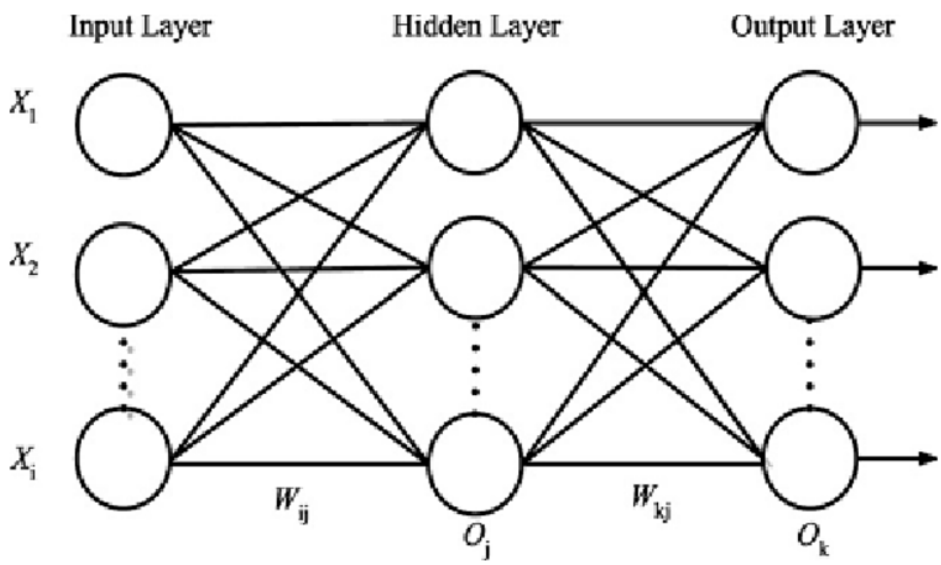
The Single Layer Feed-forward Network consists of a single layer of weights, where the inputs are directly connected to the outputs, via a series of weights. The synaptic links carrying weights connect every input to every output, but no other way. The sum of the products of the weights and the inputs is calculated in each neuron node, and if the value is above some threshold (typically 0) the neuron fires and takes the activated value (typically 1); otherwise it takes the deactivated value (typically -1).



Multi-Layer Feed Froward Network

The name suggests, it consists of multiple layers. The architecture of this class of network, besides having the input and the output layers, also have one or more intermediary layers called hidden layers. The computational units of hidden layer are known as hidden neuron.

RGPVNOTES.IN



Learning Rule:

Basically, learning means to do and adapt the change in itself as and when there is a change in environment. ANN is a complex system or more precisely we can say that it is a complex adaptive system, which can change its internal structure based on the information passing through it.

Learning rule or Learning process is a method or a mathematical logic. It improves the Artificial Neural Network's performance and applies this rule over the network. Thus learning rules updates the weights and bias levels of a network when a network simulates in a specific data environment.
Applying learning rule is an iterative process. It helps a neural network to learn from the existing conditions and improve its performance.

Hebbian Learning Rule

This rule, one of the oldest and simplest, was introduced by Donald Hebb in his book The Organization of Behavior in 1949. It is a kind of feed-forward, unsupervised learning.

Basic Concept – this rule is based on a proposal given by Hebb, who wrote –
"When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased."

From the above postulate, we can conclude that the connections between two neurons might be strengthened if the neurons fire at the same time and might weaken if they fire at different times.

Mathematical Formulation – According to Hebbian learning rule, following is the formula to increase the weight of connection at every time step.
$\Delta w_{ji}(t) = \alpha x_i(t).y_j(t)$

Here, $\Delta w_{ji}(t)$= increment by which the weight of connection increases at time step t α = the positive and constant learning rate

$x_i(t)$= the input value from pre-synaptic neuron at time step t $y_i(t)$= the output of pre-synaptic neuron at same time step t

Perceptron Learning Rule

This rule is an error correcting the supervised learning algorithm of single layer feed forward networks with linear activation function, introduced by Rosenblatt.

Basic Concept – As being supervised in nature, to calculate the error, there would be a comparison between the desired/target output and the actual output. If there is any difference found, then a change must be made to the weights of connection.

Mathematical Formulation – To explain its mathematical formulation, suppose we have 'n' number of finite input vectors, x(n), along with its desired/target output vector t(n), where n = 1 to N.

Now the output 'y' can be calculated, as explained earlier on the basis of the net input, and activation function being applied over that net input can be expressed as follows –

$y=f(y_{in})=\{1,y_{in}>\theta;\ 0,\ y_{in}\leqslant\theta$ Where θ is threshold.

The updating of weight can be done in the following two cases –

Case I – when t ≠ y, then w(new)=w(old)+tx

Case II – when t = y, then No change in weight

Perceptron Learning:

In machine learning, the perceptron is an algorithm for supervised learning of binary classifier. It is a type of linear classifier, i.e. a classification algorithm that makes its predictions based on a linear predictor function combining a set of weights with the feature vector. Perceptron network come under single-layer feed-forward networks.

The key points to be noted:

The perceptron network consists of three units, namely, sensory unit (input unit), associator unit (hidden unit), and response unit (output unit).
The sensory unit are connected to associator units with fixed weights having values 1, 0, or -1, which are assigned at random.
The binary activation function is used in sensory unit and associator unit.
The response unit has an activation function of 1, 0, or -1. The binary step with fixed threshold θ is sued as activation for associator. The output signals that are sent from the associator unit to the response time are only binary.
The output of the perceptron network is given by $y=f(y_{in})$ Where $f(y_{in})$ is activation function and is defined as:

$f(y_{in})$	=	1	If $y_{in} > \theta$
		0	If $-\theta <= y_{in} <= \theta$
		-1	If $y_{in} < -\theta$

Perceptron Network

Perceptron Learning Rule:

In case of the perceptron learning rule, the learning signal is the difference between the desired and actual response of a neuron.
Consider a finite "n" number of input training vectors, with their associated target (desired) values x(n) and t(n), where "n" ranges from 1 to N. The target is either +1 or -1. The output "y" is obtained on the basis of the net input calculated and activation function being applied over the net input.

$f(y_{in})$	=	1	If $y_{in} > \theta$
		0	If $-\theta <= y_{in} <= \theta$
		-1	If $y_{in} < -\theta$

Perceptron Learning Rule

In original perceptron network, the output obtained from the associator unit is a binary vector, and hence the output can be taken as input signal to the response unit, and classification can be performed. Here only weights between the associator unit and the output unit can be adjusted, and the weights between the sensory and associator units are fixed.

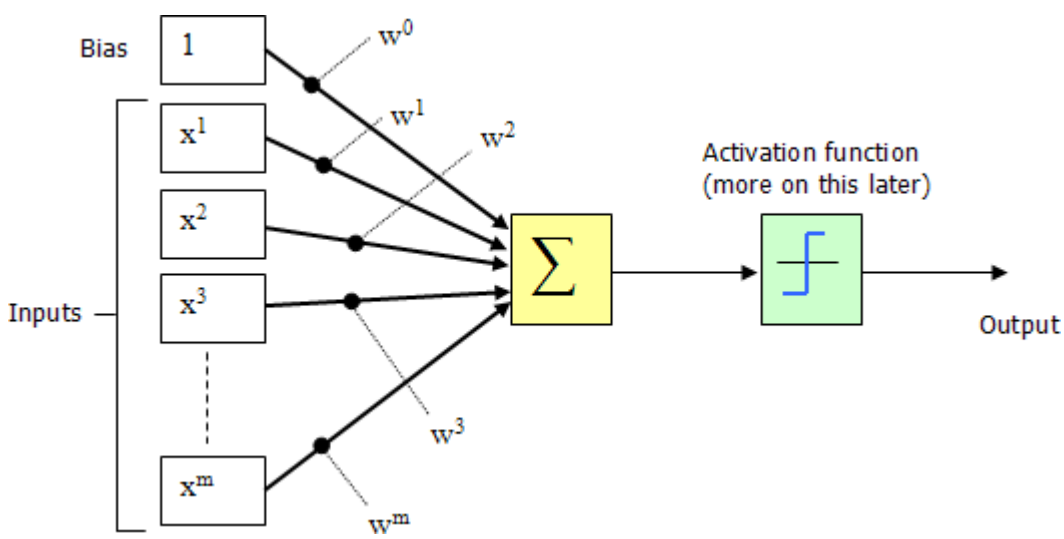


Figure: Perceptron Network

In above figure, there are n input neurons, 1 output neuron and a bias. The input-layer and output-layer neurons are connected through a directed communication link, which is associated with weights. The goal of the perceptron net is to classify the input pattern as a member or not a member to a particular class.

Perceptron Training Algorithm for Single Output Class:

The perceptron algorithm can be used for either binary input vectors, having bipolar targets, threshold being fixed and variable bias. In the algorithm, initially the inputs are assigned, then the net input is calculated. The output of the network is obtained by applying the activation

function over the calculated net input. On performing comparison over the calculated and the desired output, the weight updation process is carried out. The entire network is trained based on the mentioned stopping criteria.

The algorithm of a perceptron network is as follows:

Step 0: Initialize the weights and the bias (for easy calculation they can be set to zero). Also initialize the learning rate α ($0 < \alpha <= 1$). For simplicity α is set to 1.

Step 1: Perform steps 2-6 until the final stopping condition is false.
Step 2: Perform steps 3-5 for each training pair indicated by s:t.
Step 3: The input layer containing input units is applied with identity activation function: $x_i = s_i$
Step 4: Calculate the output of the network. To do so, first obtain the net input: n

$$Y_{in} = b + \sum_{i=1} x_i w_i$$

where "n" is the number of input neurons in the input layer. Then apply activation over the net input calculated to obtain the output:

			1	if $y_{in} > \theta$
$y = f(y_{in})$	=	{	0	if $-\theta <= y_{in} <= \theta$
			-1	if $y_{in} < -\theta$

Step 5: Weight and bias adjustment: Compare the value of the actual (calculated) output and desired (target) output.

If y != t, then
w_i (new) = w_i (old) + $\alpha t x_i$
b (new) = b(old) + αt
else, we have
w_i (new) = w_i (old)
b (new) = b (old)

Step 6: Train the network until there is no weight change. This is the stopping condition for the network. If this condition is not met, then start again form step 2.

Perceptron Training Algorithm for Multiple Output Class:

For multiple output classes, the perceptron training algorithm is as follows:
Step 0: Initialize the weights, biases and learning rate suitably.
Step 1: Check for stopping condition; if it is false, perform step 2-6.
Step 2: Perform step 3-5 for each bipolar or binary training vector pair s:t.
Step 3: Set activation (identity) of each input unit i=1 to n: $x_i = s_i$
Step 4: Calculate output response of each output unit j=1 to m: First, the net input is calculated as n

$$Y_{inj} = b_j + \sum_{i=1} x_i w_{ij}$$

The activation are applied over the net input to calculate the output response:

		{	1	if $y_{inj} > \theta$
$y_j = f(y_{inj})$	=		0	if $-\theta <= y_{inj} <= \theta$
			-1	if $y_{inj} < -\theta$

Step 5: Make adjustment in weight and bias for j=1 to m and i=1 to n If tj != yj, then

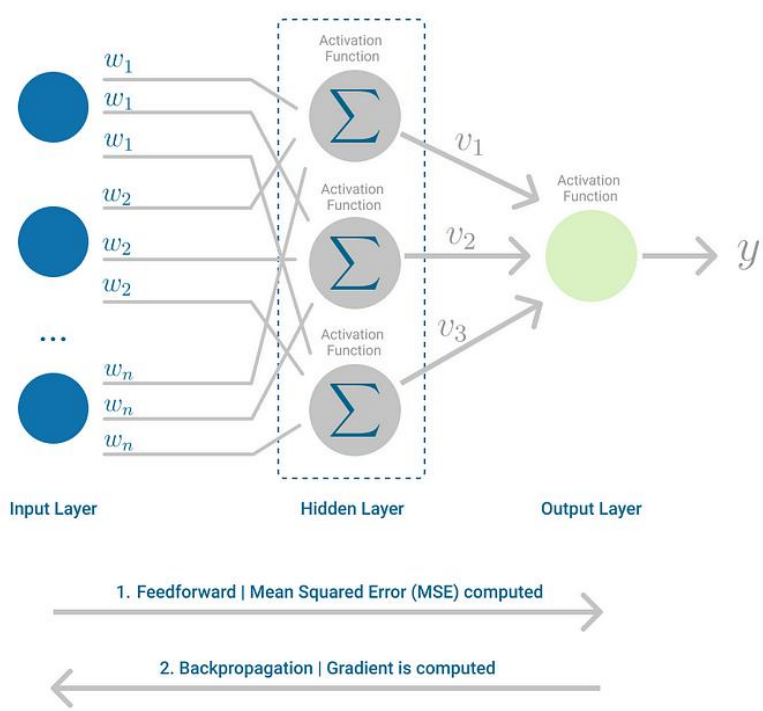
w_{ij} (new) = w_{ij} (old) + $\alpha t_j x_i$
b_j (new) = b_j (old) + αt_j
else, we have
w_{ij} (new) = w_{ij} (old)
b_j (new) = b_j (old)

Step 6: Test for the stopping condition, if there is no change in weights then stop the training process, else start again from step 2.

Backpropagation

Backpropagation is the learning mechanism that allows the Multilayer Perceptron to iteratively adjust the weights in the network, with the goal of minimizing the cost function.

There is one hard requirement for backpropagation to work properly. The function that combines inputs and weights in a neuron, for instance the weighted sum, and the threshold function, for instance ReLU, must be differentiable. These functions must have a **bounded derivative**, because Gradient Descent is typically the optimization function used in MultiLayer Perceptron.



Multilayer Perceptron, highlighting the Feedfoward and Backpropagation steps. (Image by author)

In each iteration, after the weighted sums are forwarded through all layers, the gradient of the **Mean Squared Error** is computed across all input and output pairs. Then, to propagate it back, the weights of the first hidden layer are updated with the value of the gradient. That's how the weights are propagated back to the starting point of the neural network!

$$\underbrace{\Delta_w(t)}_{\text{Gradient Current Iteration}} = -\overset{\text{Bias}}{\varepsilon}\underbrace{\frac{\overset{\text{Error}}{dE}}{dw_{(t)}}}_{\text{Weight vector}} + \overset{\text{Learning Rate}}{\alpha}\underbrace{\Delta_{w(t-1)}}_{\text{Gradient Previous Iteration}}$$

One iteration of Gradient Descent. (Image by author)

This process keeps going until gradient for each input-output pair has converged, meaning the newly computed gradient hasn't changed more than a specified *convergence threshold*, compared to the previous iteration.

<u>Back Propagation algorithm</u>:

- This algorithm Is used effectively to train a neural network through a method called chain rule.
- It is one of the most important developments in neural networks.
- Given a set of inputs and output pairs this algorithm provides a procedure for changing the weights in a back propagation network.
- Basic concept used for weight updates is gradient descent method.
- In this error is propagated back to the hidden layer.

<u>Training algorithm</u>

- Initialize weights, biases and learning rate (random small values are taken)
- Calculate net input at each hidden unit z_j j=1,2,…,p using the formula

  $(z_{in})_j = x_{0j} + \sum_{i=1}^{n} xjyij$

- Calculate outputs from hidden layers by applying activation function over $(z_{in})_j$, $z_j = f((z_{in})_j)$
- These signals are sent as input signals to the output units
- For each output unit y_k k=1,2,…,m calculate net inputs
- $(y_{in})_k = w_{0k} + \sum_{j=1}^{p} zjwjk$
- Apply activation function to compute output $Y_k = f((y_{in})_k)$ k=1,2,…,m
- Each output unit y_k k=1,2,…,m receives target pattern to the input training pattern and computes the error correction using $\Delta_k = (t_k - y_k)(f'(y_{in})_k)$
- On the basic of calculated error correction term update the change in weights and biases

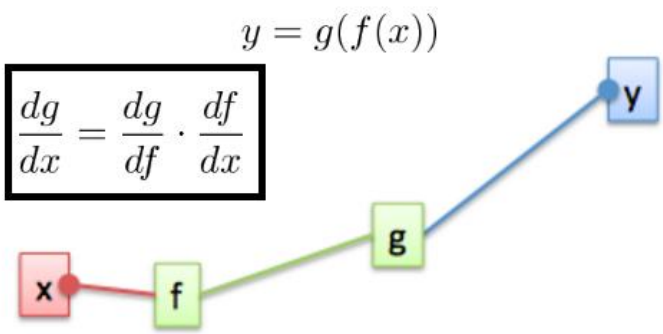
  $\Delta w_{jk} = \alpha\delta_k z_j$, $\Delta w_{0k} = \alpha\delta_k$ α is the learning rate

Or $w^{i+1} = w^{i} + (-\eta(\partial f/\partial w))$ where η is the learning rate

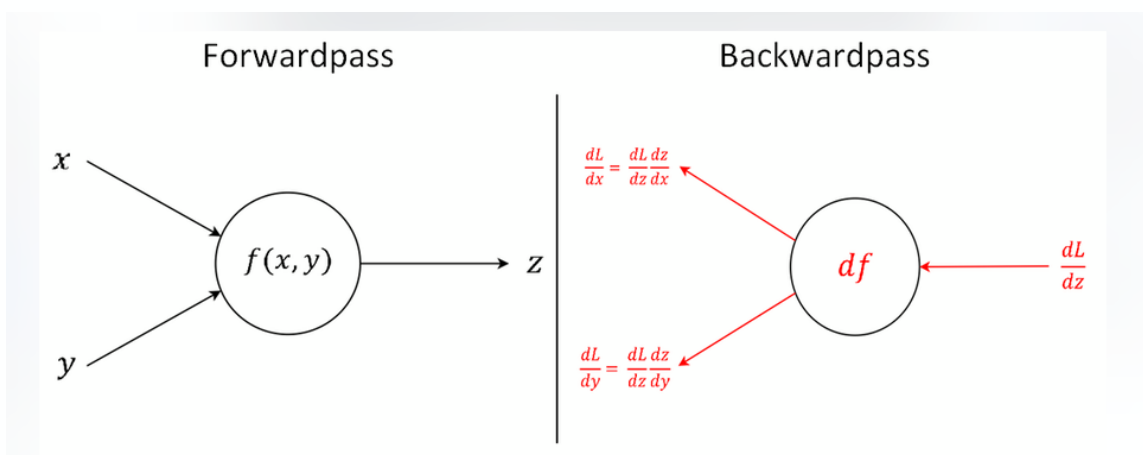
- Also send δ_k to the hidden layer backward
- Update weights and biases for the output and hidden units
  $W_{jk}(new) = W_{jk}(old) + \Delta w_{jk}$
  $W_{0k}(new) = W_{0k}(old) + \Delta w_{0k}$

Chain Rule

Imagine that you have an output y, that is function of g, which is function of f, which is function of x. If you want to know how much g will change with a small change on dx (dg/dx), we use the chain rule. Chain rule is a formula for computing the derivative of the composition of two or more functions.

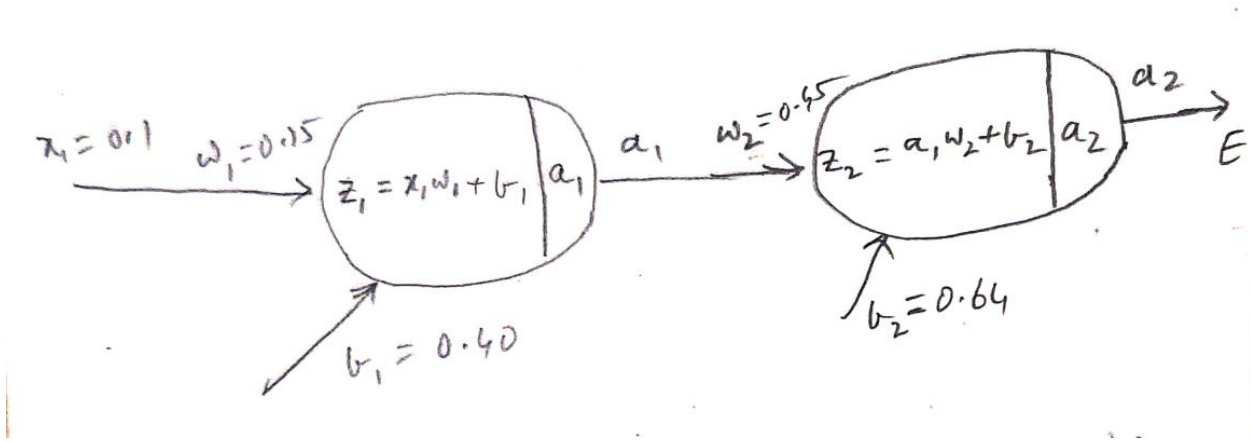


The chain rule is the work horse of back-propagation, so it's important to understand it now. On the picture bellow we get a node f(x,y) that compute some function with two inputs x,y and output z. Now on the right side, we have this same node receiving from somewhere (loss function) a gradient dL/dz which means. "How much L will change with a small change on z". As the node has 2 inputs it will have 2 gradients. One showing how L will a small change dx and the other showing how L will change with a small change (dz)



In order to calculate the gradients we need the input dL/dz (dout), and the derivative of the function f(x,y), at that particular input, then we just multiply them. Also we need the previous cached input, saved during forward propagation.

Numerical example for back propagation



Activation function used is sigmoid

$Z_1 = x_1w_1+b_1$ $\quad a_1 = f(z_1) = (1/(1+e^{-z1}))$ $\quad Z_2 = a_1w_2+b_2$ $\quad a_2 = f(z_2) = (1/(1+e^{-z2}))$

Error = $E= (1/2)(t-a_2)^2$

To update weight w_1

- $\partial E/\partial w_1 = \partial E/\partial a_2 * \partial a_2/\partial z_2 * \partial z_2/\partial a_1 * \partial a_1/\partial z_1 * \partial z_1/\partial w_1$ (Chain rule)
- $\partial z_2/\partial a_1 = \partial(a_1w_2+b_2)/\partial w_2 = w_2$
- $\partial a_1/\partial z_1 = \partial(1/(1+e^{-z1})) / \partial z_1 = f(z_1) * (1- f(z_1))= a_1(1-a_1)$
- $\partial a_2/\partial z_2 = \partial(1/(1+e^{-z2})) / \partial z_2 = f(z_2) * (1- f(z_2))= a_2(1-a_2)$
- $\partial z_1/\partial w_1 = \partial(x_1w_1+b_1)/\partial w_1 = x_1$
- $\partial E/\partial w_1 = \partial E/\partial a_2 * \partial a_2/\partial z_2 * \partial z_2/\partial a_1 * \partial a_1/\partial z_1 * \partial z_1/\partial w_1 = -(t-a_2)* a_2(1-a_2) * w_2 * a_1(1-a_1) * x_1$
- $\partial E/\partial b_1 = \partial E/\partial a_2 * \partial a_2/\partial z_2 * \partial z_2/\partial a_1 * \partial a_1/\partial z_1 * \partial z_1/\partial b_1$
- $\partial E/\partial b_1 = -(t-a_2)* a_2(1-a_2) * w_2 * a_1(1-a_1) * 1$

- Consider epochs = 100 threshold = 0.001 learning rate (α) = 0.4 t=0.25
- z1 =x1 * w1 + b1 = 0.415 z2 = a1 * w2 + b2 = 0.9210
- a1= 1/1(1+e-0.415) = 0.6023
- a2= 1/1(1+e-0.9210) = 0.7153
- Therefore

  z1 = 0.415 z2 = 0.9210 a1 = 0.6023, a2= 0.7153
- Error = $½(t-a2)^2$ $½(0.25 -0.7153)^2$ =0.1083
- <u>Updating w2</u>

  $w2(new)=w2(old) - \alpha\partial E/\partial w_2$

  = 0.45 – 0.4(-(0.25 -0.7153))* 0.7153*(1-0.7153)*0.6023

  = 0.427
- <u>Update b2</u>

  $b2(new)=b2(old)- \partial E/\partial b_2$

  =0.65 –(t-a2)*a2(1-a2)*1

  = 0.612

W1(new)=w1(old)+ $\alpha\partial E/\partial w1$ = 0.1496

b1(new)=b1(old)- $\partial E/\partial b1$ = 0.3959

We continue next iteration (feed forward) with the updated weights and biases.

W1=0.1496 w2 =0.427 b1=0.612 b2=0.3959.

These iterations will be done till the error is minimized to the given threshold.